

System Architecture

From Kaggle Model Training to the VS Code Web Application

This document explains, end to end, how the thesis pipeline is built: models are trained on Kaggle's free GPU, their results are imported into a local FastAPI + Postgres + MinIO backend, mirrored to a plain folder tree for the thesis write-up, and finally displayed and reused by a browser-based web application built and run from VS Code.

How the system fits together: Kaggle training → local storage → web application

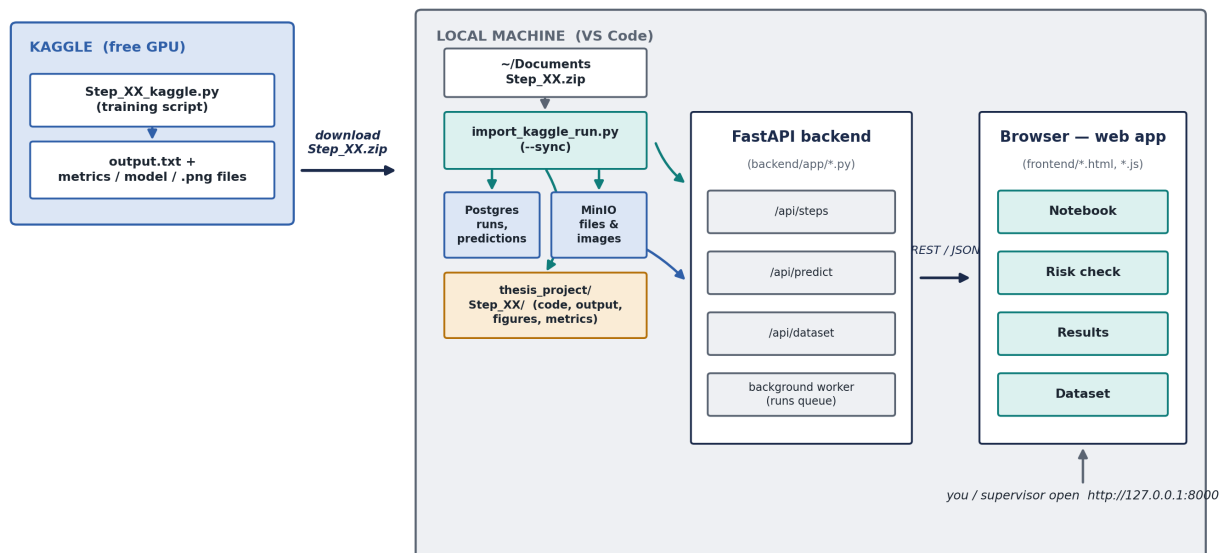


Figure 1. The three environments — Kaggle, the local machine, and the browser — and how data moves between them.

1. Why training and the web app live in different places

The three models compared in this thesis have very different compute needs. CatBoost trains in seconds on a CPU. TabPFN and TabFM are pretrained tabular foundation models: they need a GPU to fit the full training set in a reasonable time and, for TabFM in particular, to compute SHAP explanations at all (a single SHAP run at the largest context took over 2 hours — 8,491 seconds — even on Kaggle's GPU, see [Step_10_FinalResults](#)). The laptop this app runs on is CPU-only, so every heavy step — training and SHAP for the foundation models — is done once on Kaggle, and only the results (metrics, figures, the trained CatBoost model, log files) travel back to the laptop.

The web application itself is deliberately simple: a FastAPI backend, a Postgres database, a MinIO file store and a plain HTML/JavaScript frontend, all started with one script (`./start.sh`) and editable directly in VS Code. Its job is not to retrain models but to organise, display and reuse what Kaggle already produced.

2. Component roles

Component	Role
Kaggle notebook	Runs the GPU-heavy step scripts (<code>backend/kaggle/Step_XX_..._kaggle.py</code>) and produces <code>output.txt</code> , <code>metrics.json</code> and any figures/models for that step.
<code>import_kaggle_run.py</code>	The bridge. Matches a downloaded Kaggle result folder to a step by id, pairs it with the local copy of that step's code, and loads everything into Postgres + MinIO.
Postgres ("runs" table)	One row per executed step (local or Kaggle), holding its code, status, console output and timing.
MinIO	Object storage for every file a step produces — images, model files, CSVs — addressed by key, same as a small local S3.
<code>thesis_project/<Step>/</code>	A plain folder mirror of the latest successful run of each step: <code>code.py</code> , <code>output.txt</code> , figures and metrics. This is what the thesis appendix and any manual inspection point to.
FastAPI backend (<code>backend/app/*.py</code>)	Serves the REST API the frontend calls: <code>/api/steps</code> , <code>/api/predict</code> , <code>/api/dataset</code> , plus a background worker that executes queued notebook runs.
Frontend (<code>frontend/*.html</code> , <code>*.js</code>)	The four-tab browser application: Notebook, Risk check, Results, Dataset. Talks to the backend only via REST/JSON.

3. One training step's journey, in detail

The diagram below traces exactly what happens to a single step — for example Step_07c_TabFM_Full — from the moment it is run on Kaggle to the moment its code and output appear in the Notebook tab.

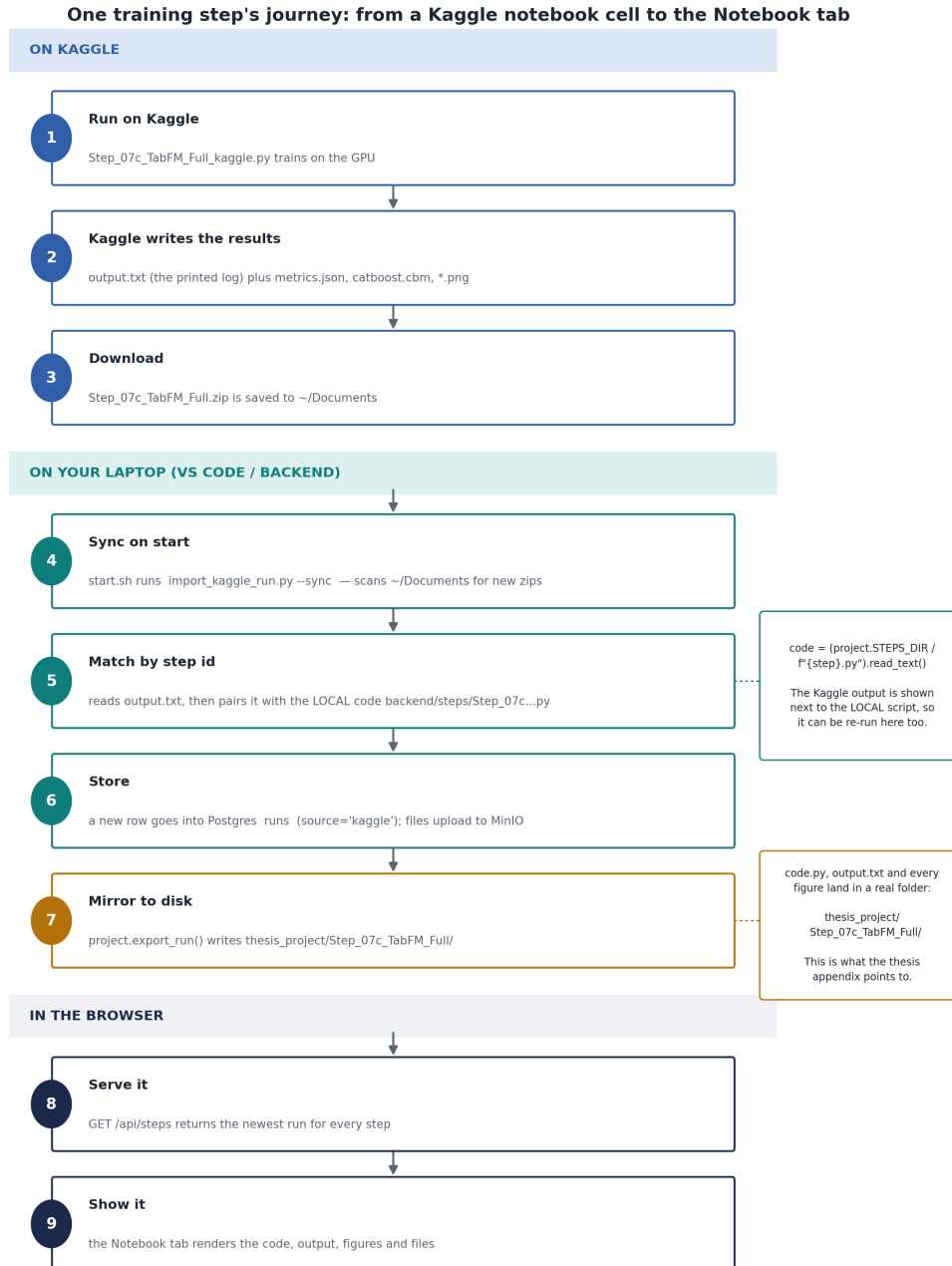


Figure 2. The nine-step journey of one Kaggle result into the Notebook tab.

Steps explained

#	Step	What actually happens
1–2	Train & export on Kaggle	The Kaggle version of the step's script runs on the GPU and prints its log to stdout; that log becomes output.txt. Any result files (metrics.json, catboost.cbm, *.png) are saved alongside it, and the folder is zipped for download.
3	Download	The zip (e.g. Step_07c_TabFM_Full.zip) is saved to ~/Documents — no manual unzip or renaming needed.
4	Sync on start	Every time the app starts, start.sh runs <code>import_kaggle_run.py --sync</code> , which scans ~/Documents for the newest zip per step and skips any step whose log has not changed — so it is always safe to run.
5	Match by step id	The importer reads output.txt from the Kaggle zip, but loads the CODE from the local file <code>backend/steps/Step_07c_TabFM_Full.py</code> — not the Kaggle variant. This means the exact code shown and re-runnable in the Notebook tab is the maintained local version, even though the output came from Kaggle.
6	Store	A new row is inserted into the Postgres runs table tagged <code>source='kaggle'</code> , and every result file is uploaded to MinIO.
7	Mirror to disk	<code>project.export_run()</code> writes the code, output and figures out to <code>thesis_project/Step_07c_TabFM_Full/</code> on disk, moving any previous run of that step into a <code>history/</code> subfolder first.
8–9	Serve & show	<code>GET /api/steps</code> returns, for every step, whichever run is newest — Kaggle or a local re-run — and the Notebook tab renders its code, console output, figures and downloadable files.

Why the code is re-paired with the local copy

A deliberate design choice: the Notebook tab always shows the LOCAL script for a step, even when the result being displayed came from Kaggle. That local script can then be edited and re-run directly from the browser (on CPU, for the smaller steps), while the heavy training runs stay on Kaggle. The relevant line is:

```
code = (project.STEPS_DIR / f"{step}.py").read_text()
```

— in `backend/import_kaggle_run.py`, function `import_step()`.

4. The web application's four tabs

Tab	What it is for
Notebook	The thesis pipeline, step by step: view or edit code, run it, see console output, figures and files, and reopen any earlier run — whether it ran locally or was imported from Kaggle.
Risk check	A person enters their lab results and lifestyle answers; the app returns a plain-language hair-fall risk assessment using the same trained models, with a Developer view showing each model's raw probabilities and the calculation.
Results	All models × training sizes side by side: accuracy/F1/ROC-AUC tables, scaling charts, McNemar's significance tests, confusion matrices, SHAP summaries and CatBoost hyperparameter tuning — the material behind the thesis's results chapter.
Dataset	The raw patient table (21,606 rows) with search, sorting and per-column statistics.

5. Closing the loop: the same models answer a live Risk check

The Risk check tab is not a separate model — it reuses exactly what Steps 4–7 produced: the Step 4 training rows (as TabPFN/TabFM's in-context examples) and the Step 5c CatBoost model file. This is the same pipeline shown above, just closing the loop by putting the trained artifacts to work on one new patient instead of a held-out test set.

The same pipeline, closing the loop: trained models answer a live Risk check

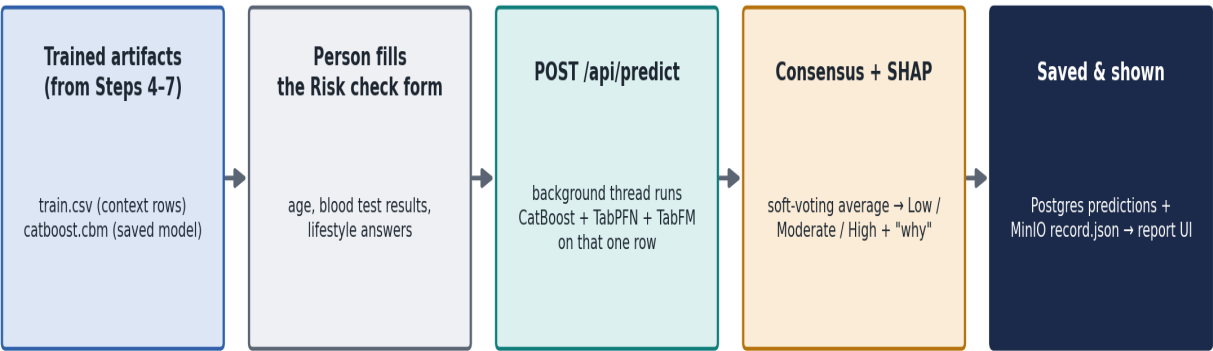


Figure 3. A Risk check request reuses the artifacts trained in Steps 4–7.

All three models run in a background thread; a soft-voting average of their class probabilities gives the final Low/Moderate/High risk and a confidence score, and CatBoost's exact TreeSHAP values are used to explain “why” (TabPFN/TabFM's KernelSHAP is orders of magnitude slower — minutes to hours, see Step 10 — so it is only computed offline, not per patient). Every check is saved to Postgres and mirrored to MinIO as a JSON record, so it can be reopened later.

6. Where things live

What	Where
App (once started)	http://127.0.0.1:8000
MinIO console	http://127.0.0.1:9001 (minioadmin / minioadmin)
Step code (editable)	backend/steps/
Kaggle-packaged step code	backend/kaggle/
Kaggle → local importer	backend/import_kaggle_run.py
Results per step (thesis appendix)	thesis_project/<Step>/
Database + object store data	backend/data/

In one sentence: each Kaggle notebook step exports its log and result files as a folder; a small importer matches each one to its local script by step id, loads it into Postgres/MinIO, and mirrors it to disk; the web app's Notebook tab displays whichever run — Kaggle or local — is current for each step, and the Risk check tab reuses the same trained artifacts to answer live predictions.